

**Московский Физико-Технический Институт
Физтех-школа Аэрокосмических технологий
Институт Аэромеханики и Летательной Техники**



ФАКТ

**Архитектура Компьютера
и Операционные Системы.
Часть 2. Основы операционных систем**

**Лекция 12: Введение в сетевое
программирование в UNIX. Часть 2/4: TCP/IP**

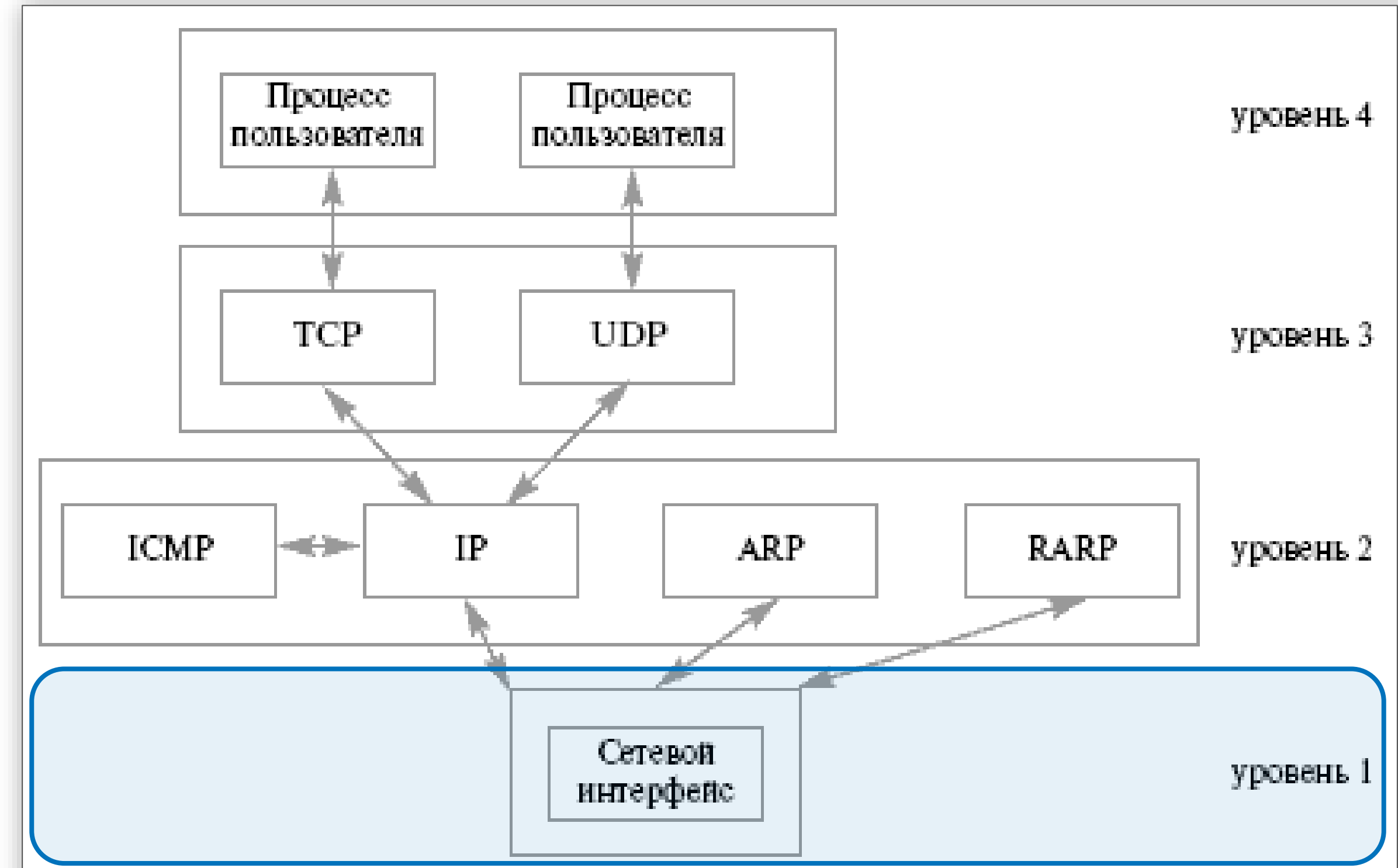
**Новиков Андрей Валерьевич
д.ф.-м.н.**

Жуковский

Структура TCP/IP

1 – Уровень сетевого интерфейса

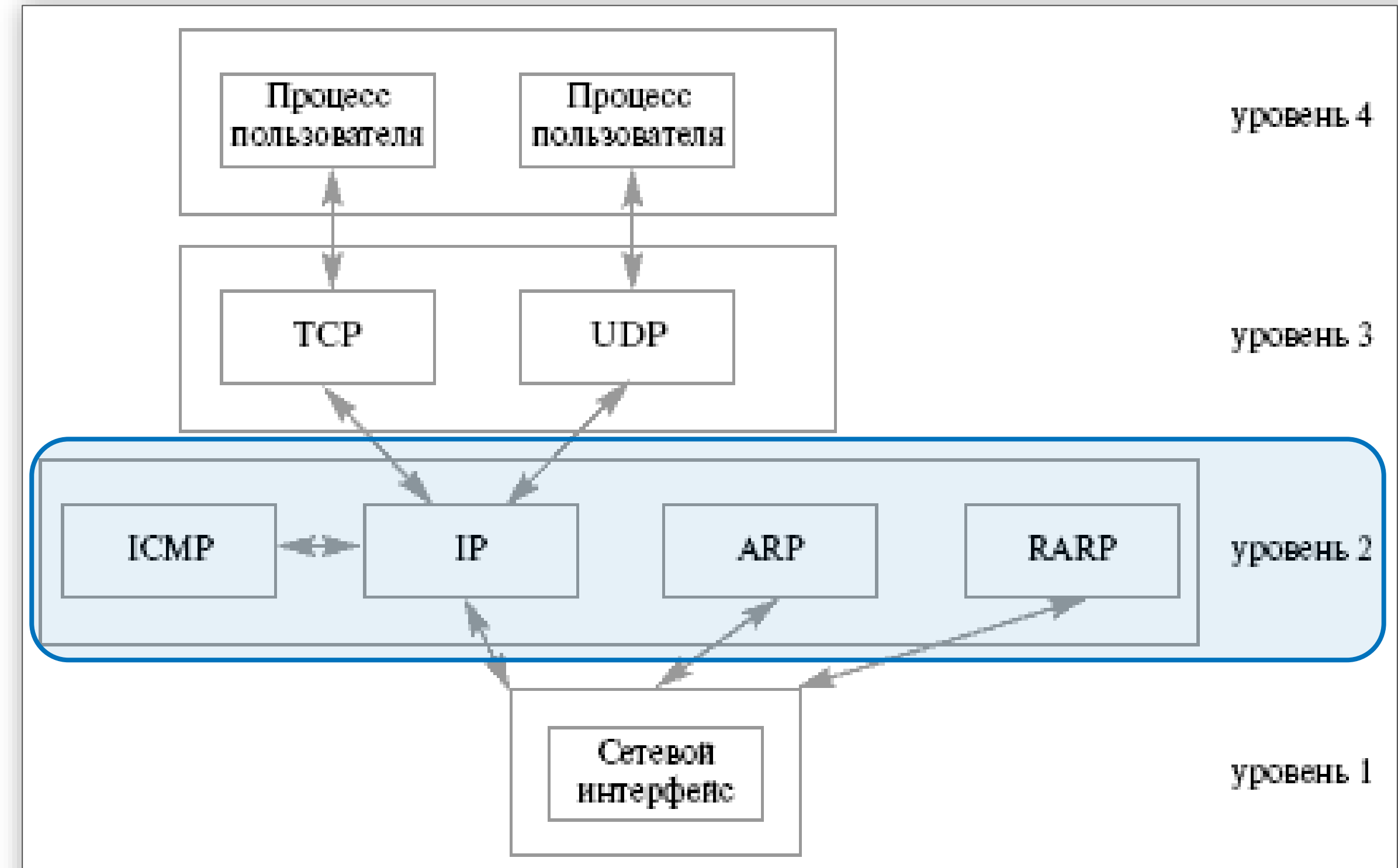
- ❑ Передача данных между узлами связи, физически напрямую соединёнными друг с другом (подключёнными к одному сегменту сети), и физические средства передачи данных.
- ❑ Протоколы Ethernet, Token Ring, SLIP, PPP и т.д.; витая пара, коаксиальный кабель, оптоволоконный кабель и т.д. Формально не относятся к TCP/IP, но определяют как осуществляется передача данных в TCP/IP с использованием этих протоколов. На уровне сетевого интерфейса в UNIX работают драйверы различных сетевых плат.
- ❑ Обмен на основании физических адресов точек входа сети в узлы связи. Каждая точка входа имеет свой уникальный адрес – MAC-адрес (Media Access Control), физически зашитый при изготовлении. Например, в Ethernet, MAC - 48-битовый номер.



Структура TCP/IP

2 – Уровень Интернет

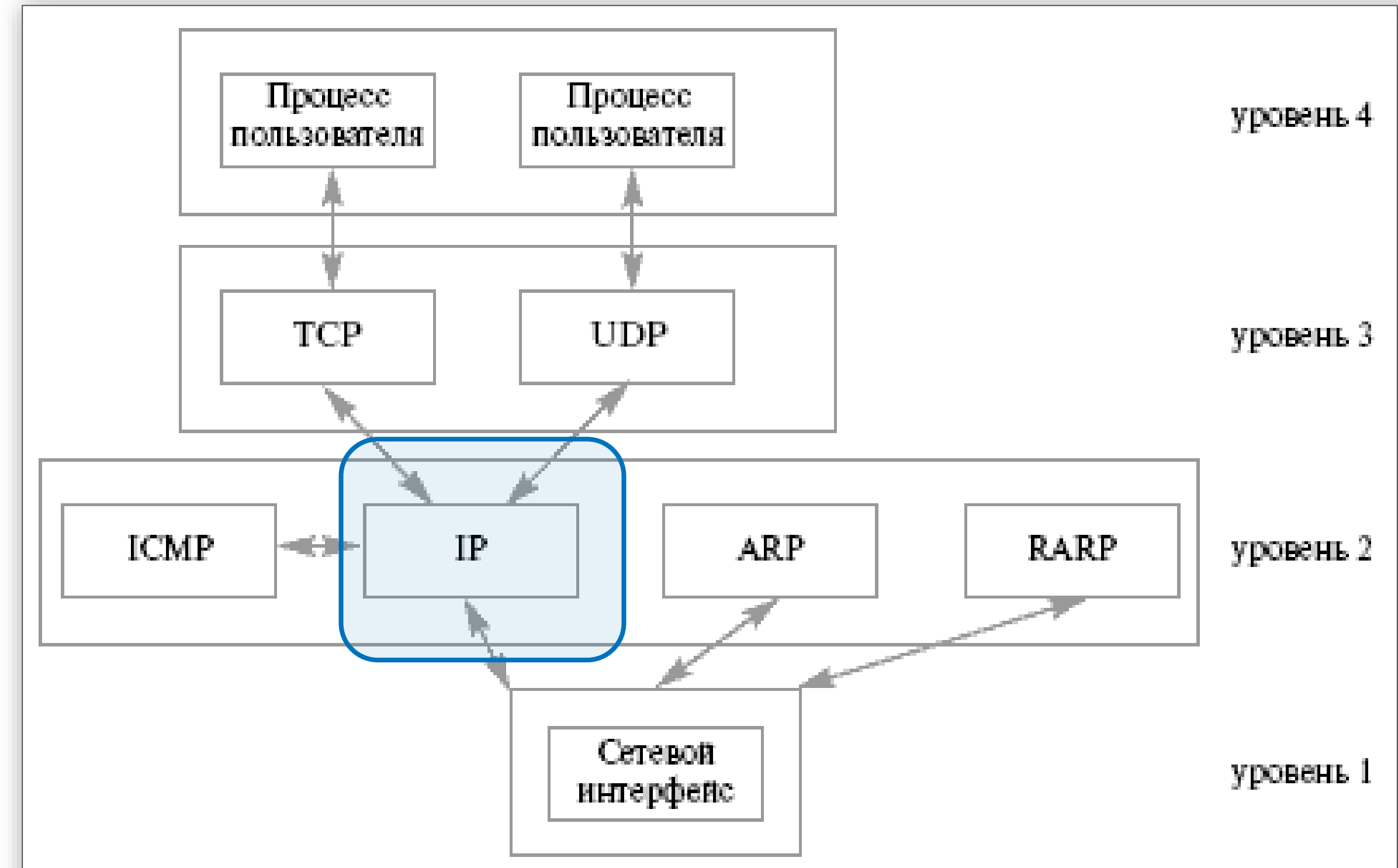
- ❑ Обеспечивает доставку информации от сетевого узла отправителя к сетевому узлу получателя без установления *виртуального соединения* с помощью *датаграмм* и не является надежным.
- ❑ Отвечает за маршрутизацию пакетов, может частично управлять скоростью передачи данных (он может попросить отправителя уменьшить скорость передачи).
- ❑ Протоколы:
 - ICMP** (Internet Control Message Protocol). Обработка ошибок и обмен управляющей информацией.
 - IP** (**Internet Protocol**). Доставка пакетов для протокола **ICMP** и протоколов транспортного уровня TCP и UDP.
 - ARP** (Address Resolution Protocol),
 - RARP** (Reverse Address Resolution Protocol) - Отображение адресов уровня *Internet* в адреса уровня сетевого интерфейса и обратно



Структура TCP/IP

2 – Уровень Интернет. IP протокол

- ❑ Центральным протокол уровня. Вся информация, поступающая к нему от других протоколов, оформляется в виде *IP-пакетов данных (IP datagrams)*.
- ❑ *IP-пакет* содержит адреса компьютеров отправителя и получателя, поэтому может передаваться независимо от других пакетов *по* своему собственному маршруту. Любая *связь* между пакетами, предполагающая знания об их содержании, должна осуществляться на более высоком уровне стека протоколов.
- ❑ *IP-уровень не обеспечивает надежную связь*, не гарантирует доставку и/или отсутствие ошибок. *IP* проверяет контрольную сумму только заголовка пакета (адреса отправителя, получателя, etc). Если *IP*-заголовок пакета испорчен, то весь пакет просто отбрасывается, повторная передача возлагается на вышестоящие уровни.

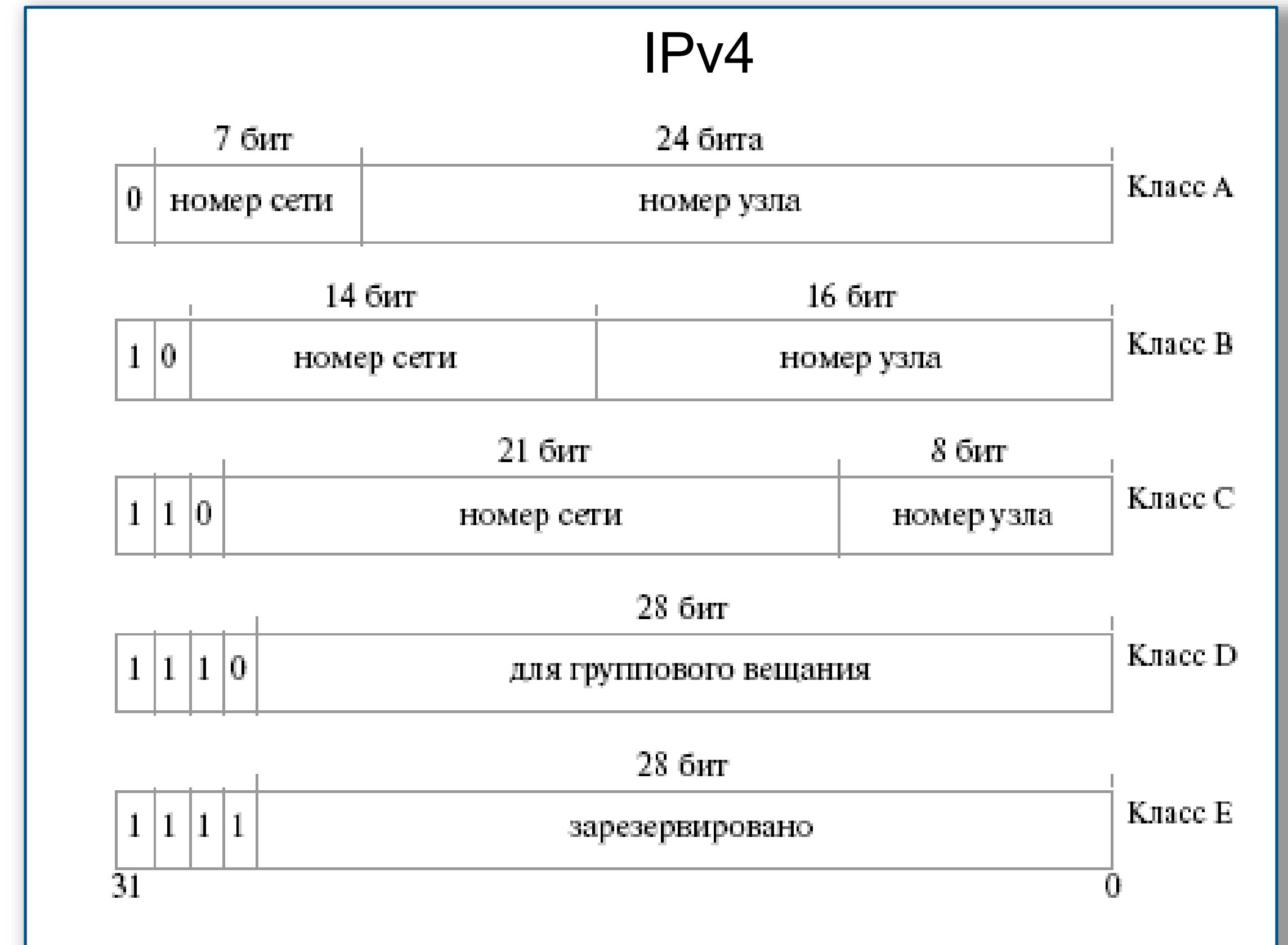


- ❑ *IP протокол* делает фрагментацию и дефрагментацию данных. Слишком большие пакеты разбиваются на фрагменты, каждый в виде нового *IP*-пакета с теми же адресами отправителя и получателя. Дефрагментация у получателя. Если хоть один фрагмент потерян, то отбрасывается и весь пакет целиком.

IP адреса

2 – Уровень Интернет. Адресация

- ❑ IP-адрес точки подключения компьютера к сети – часть полных адресов удалённых процессов
- ❑ IP версии 4 (IPv4): 32-битные адреса
IP версии 6 (IPv6): 128-битные адреса.
- ❑ IPv4-адреса делятся на 5 классов по количеству последовательных 1 в старших битах.
А, В и С - адресации сетевых интерфейсов;
D – групповая рассылка (*multicast addresses*);
E – был зарезервирован для будущих расширений.
- ❑ IP-адрес класса А–С = идентификатор или №сети + идентификатор или номер узла в этой сети.
№ сети присваивается **ICANN** (Корпорация Internet по присвоению имен и номеров); № узла присваивается администратором сети
- ❑ IP-адреса для удобства записывают каждый байт по отдельности в десятичном виде через точку:
192.168.253.10.

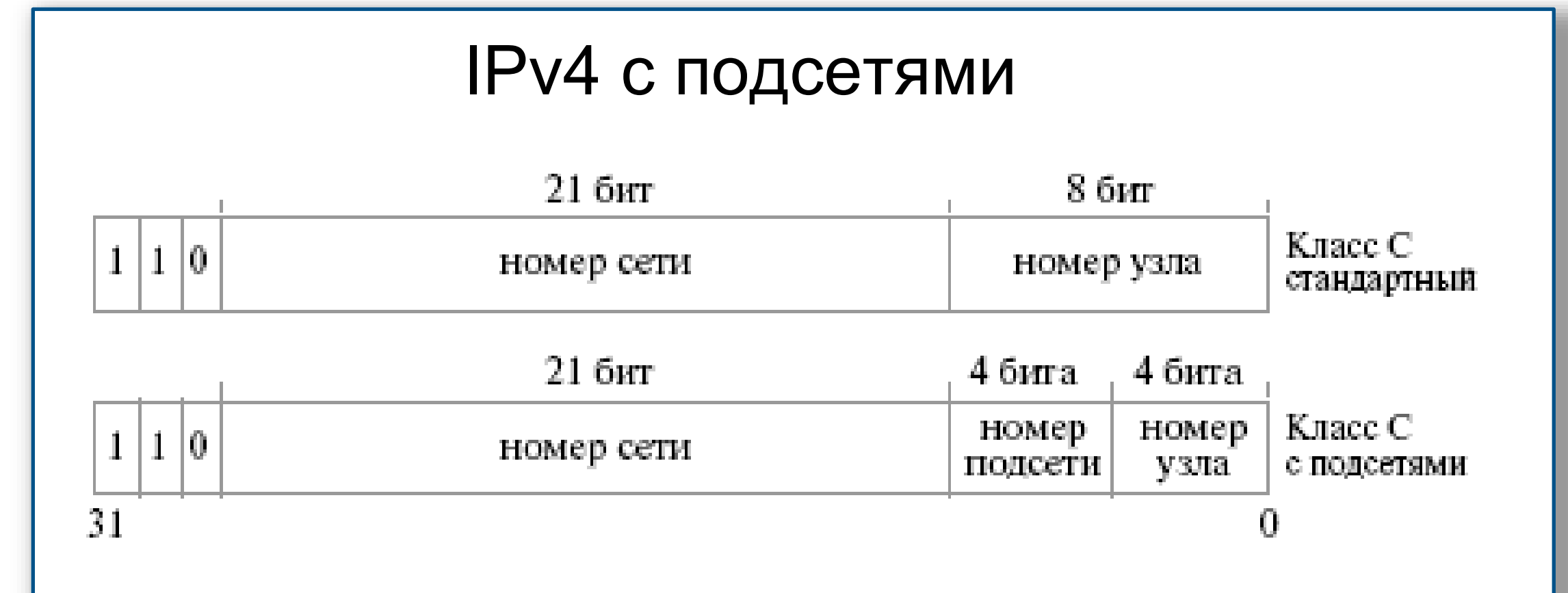


- ❑ Класс А - небольшое количество сетей, содержащих очень много компьютеров
- ❑ Класс С – большое количества сетей с малым числом компьютеров.
- ❑ Все идентификаторы сетей к настоящему моменту уже задействованы.

IP адреса

2 – Уровень Интернет. Адресация с подсетями

- ❑ Администратор сети любого класса, может произвольным образом разделить *адресное пространство* идентификаторов узлов на подсети.
- ❑ Несколько старших *бит* из адресного пространства идентификаторов узлов выделяется для идентификации сегмента сети (например 4 бита), а для адресации узлов внутри сегмента используются оставшиеся 4 бита: адресация с использованием подсетей.
- ❑ Маска подсети — битовая маска для определения по IP-адресу адреса подсети и адреса узла (хоста, компьютера, устройства) этой подсети.



IP-адрес: 11000000 10101000 00000000 **1 00000010** (192.168.1.2)
Маска подсети: **11111111 11111111 11111110** 00000000 (255.255.254.0)
Адрес сети: **11000000 10101000 00000000** 00000000 (192.168.0.0)

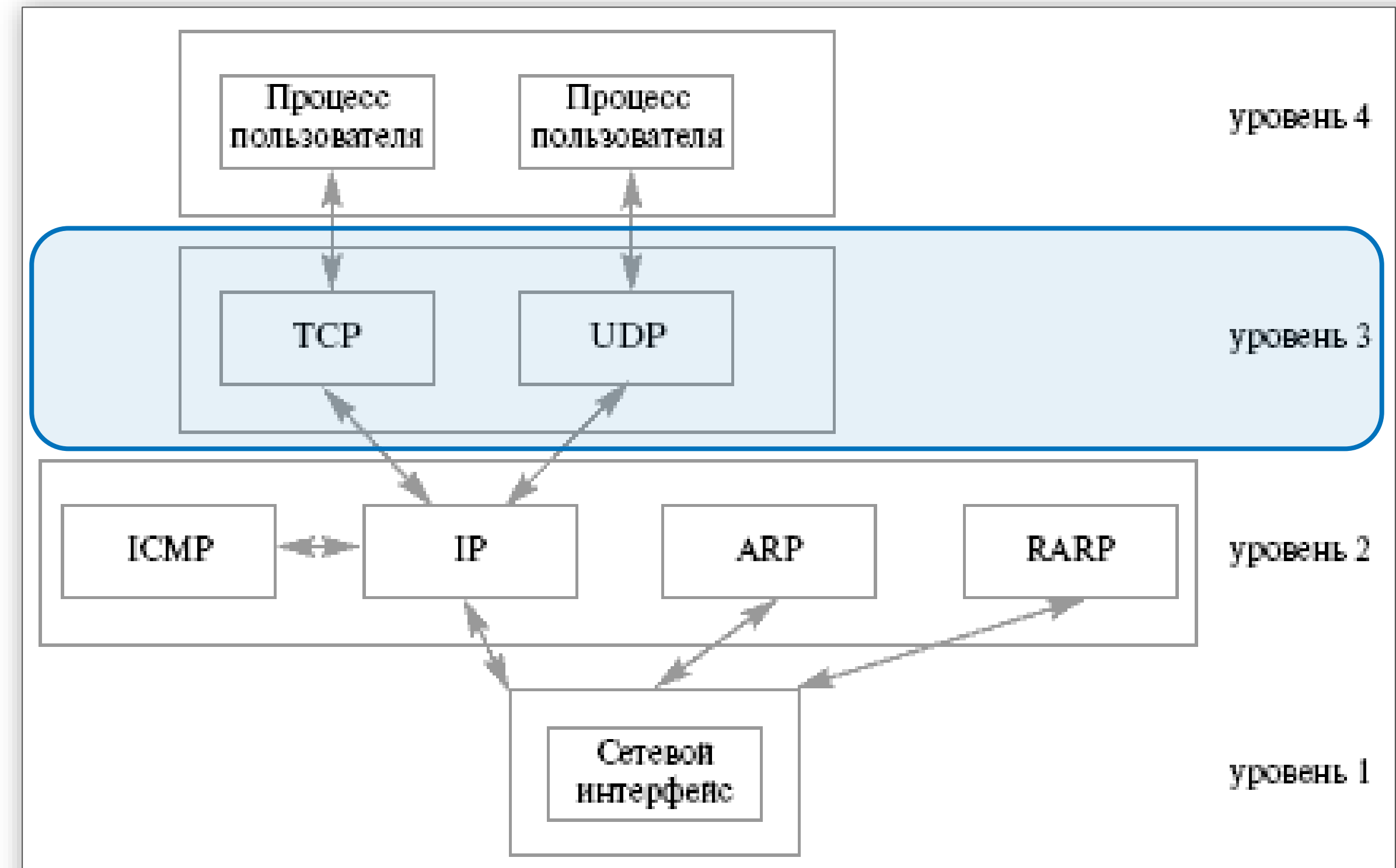
Структура TCP/IP

3 – Транспортный уровень

❑ Протоколы: TCP и UDP

❑ **TCP** реализует потоковую модель передачи информации, хотя в его основе лежит обмен информацией через пакеты данных.

- ориентированный на установление логической связи (**connection-oriented**)
- **надежный**: проверка контрольных сумм, передача подтверждения в случае правильного приема сообщения, повторная передача пакета в случае — неполучения подтверждения в течение определенного промежутка времени, правильную последовательность получения информации, полный контроль скорости передачи данных) дуплексный способ связи между процессами в сети.



Структура TCP/IP

3 – Транспортный уровень

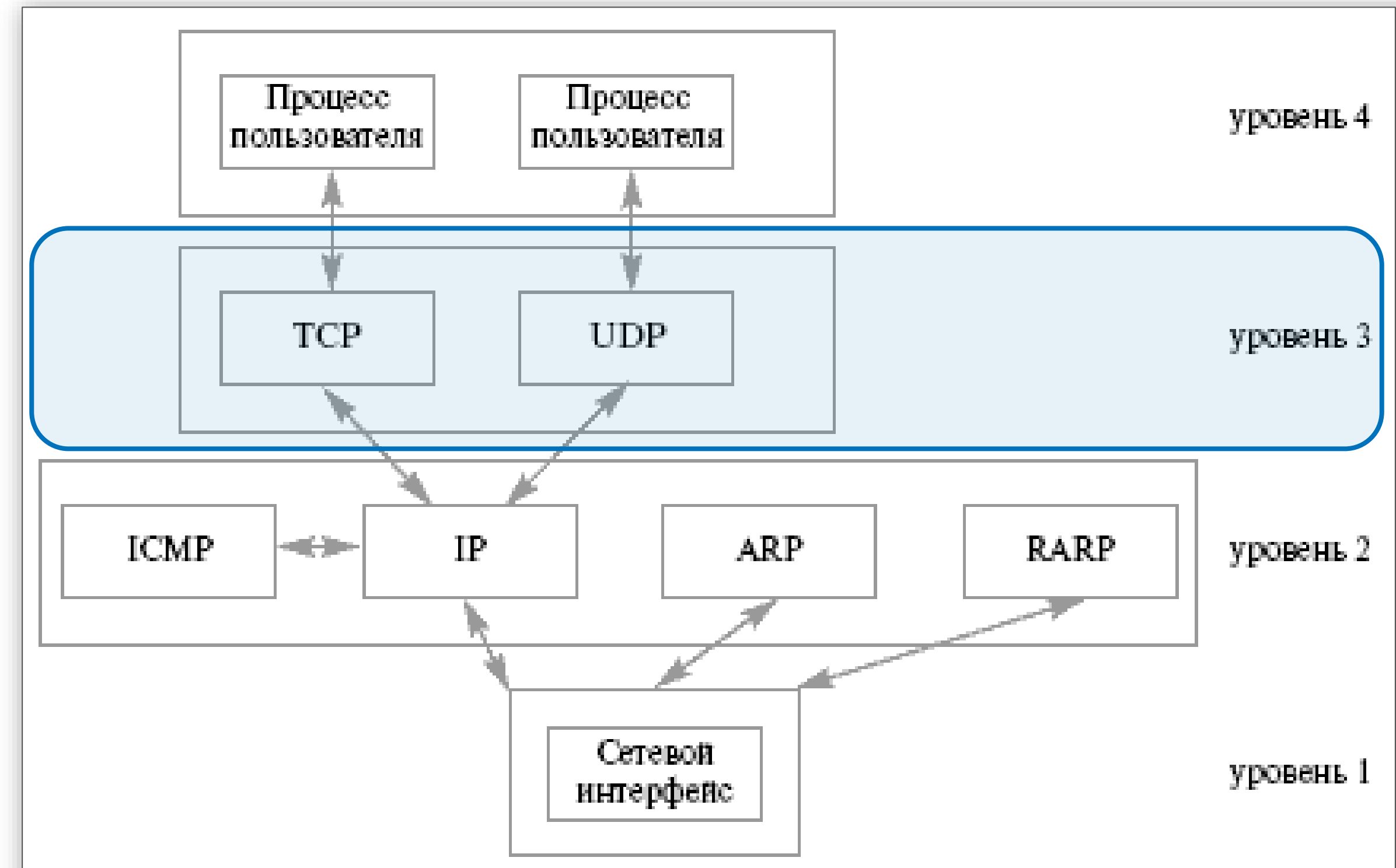
❑ Протоколы: TCP и UDP

❑ UDP

- ориентирован на передачу сообщений (*датаграмм*);
- ненадежный.

Отличается от IP-протокола на уровне 2:

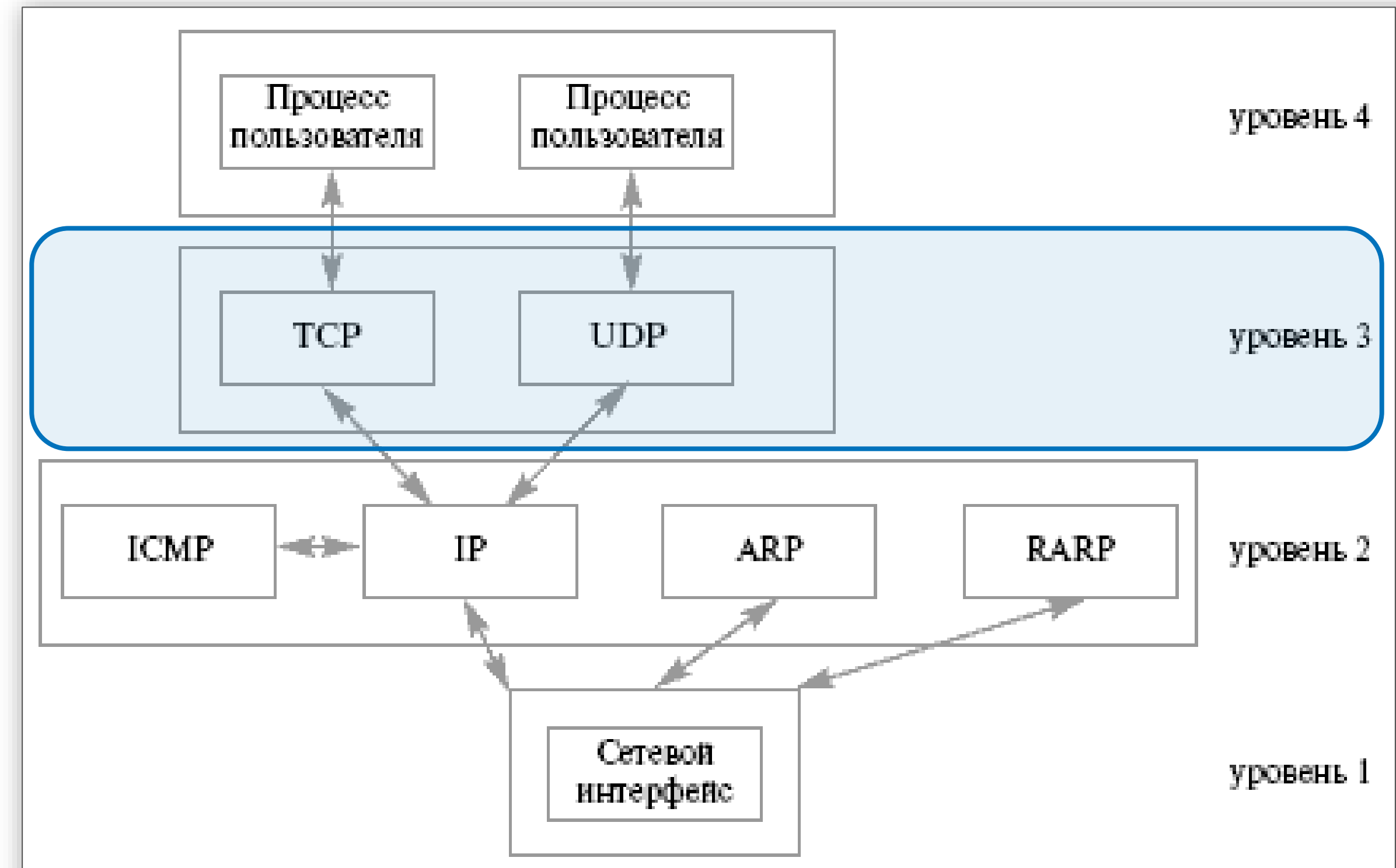
- правильность принятого сообщения проверяется контрольной суммой, насчитанной по всему сообщению;
- информация передаётся не от узла сети к другому узлу, а от отправителя к получателю.



Структура TCP/IP

3 – Транспортный уровень. Адресация

- ❑ Полный адрес удаленного процесса для конкретного способа связи определяется парой адресов:
<адрес компьютера в сети, локальный адрес>.
- ❑ Организация взаимодействия удалённых процессов через спец. объект **socket** (гнездо, панель) - виртуальный коммуникационный узел (как бы разъем или ящик для приема/отправки писем), ведущим от объекта во внешний мир и наоборот. При *непрямой адресации* сами промежуточные объекты для организации взаимодействия процессов также именуются *сокетами*.
- ❑ Полный адрес = адрес сокета
- ❑ Локальный адрес сокета = порт
Адресное пространство *портов* - положительные значения целого 16-битового числа. Различны для UDP и TCP.

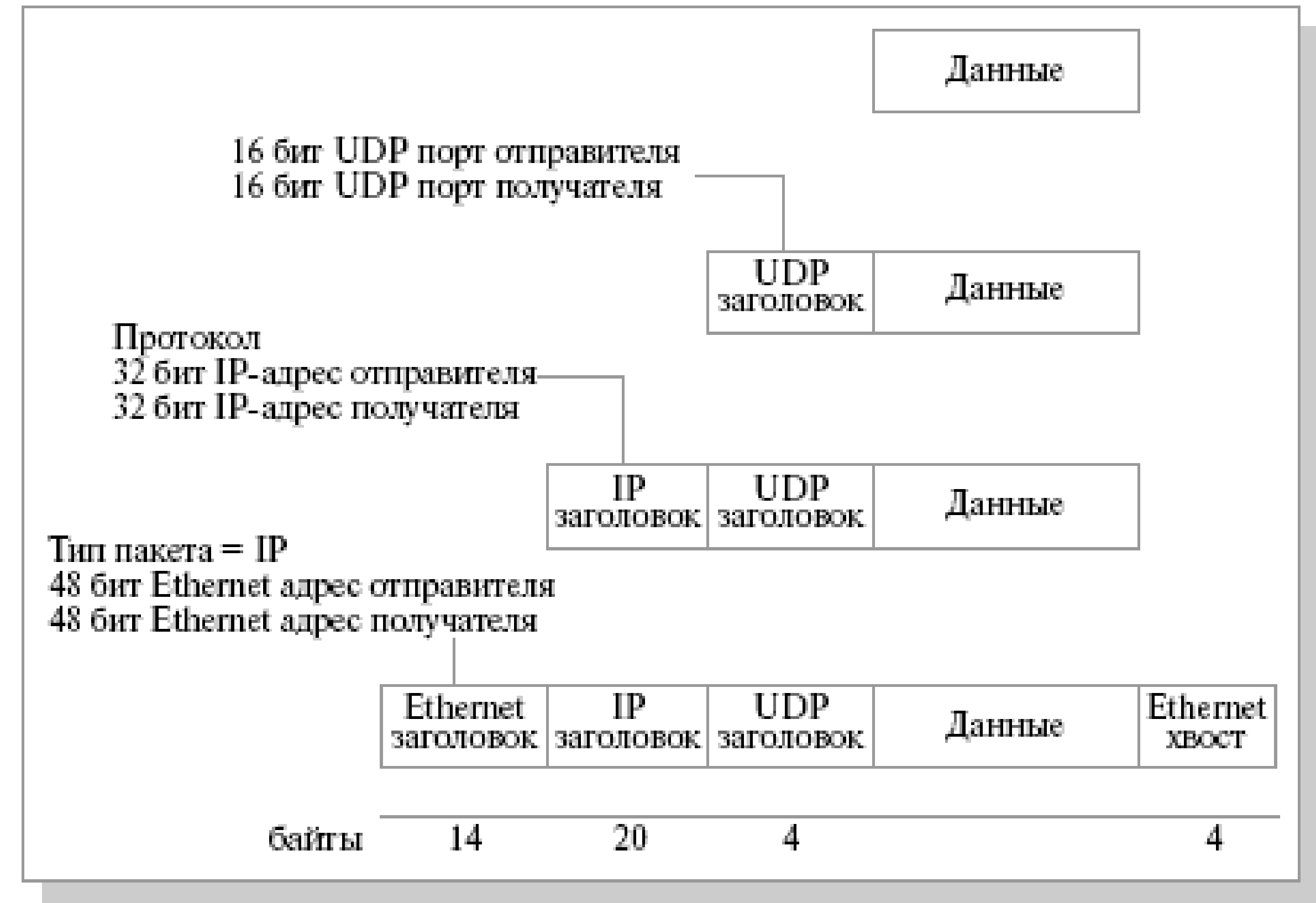


Система адресации ТСП/IP

Иерархическая система адресации стеке ТСП/IP

(encapsulation)

- ❑ Физический пакет данных, передаваемый по сети, содержит физические адреса узлов сети (MAC-адреса) с указанием на то, какой протокол уровня Internet должен использоваться для обработки передаваемых данных (IP, ICMP, ...)
- ❑ IP-пакет данных содержит 32-битовые IP-адреса компьютера-отправителя и компьютера-получателя, и указание на то, какой вышележащий протокол (TCP, UDP или еще что-нибудь) должен использоваться для их дальнейшей обработки.
- ❑ Служебная информация транспортных протоколов (UDP-заголовок к данным и TCP-заголовок к данным) должна содержать 16-битовые номера портов для сокета отправителя и сокета получателя.



Модель клиент-сервер

- ❑ Сервер, как правило, работает постоянно, на всем протяжении жизни приложения, а клиенты могут работать эпизодически.
- ❑ Сервер ждет запроса от клиентов, инициатором же взаимодействия выступает клиент.
- ❑ Как правило, клиент обращается к одному серверу за раз, в то время как к серверу могут одновременно поступить запросы от нескольких клиентов.
- ❑ Клиент должен знать полный адрес сервера (его локальную и удаленную части) перед началом организации запроса (до начала общения), в то время как сервер может получить информацию о полном адресе клиента из пришедшего запроса (после начала общения).
- ❑ И клиент, и сервер должны использовать один и тот же протокол транспортного уровня.

Связь между удаленными процессами с помощью датаграмм

UDP

Взаимодействие клиента и сервера для протокола UDP

Процесс-сервер:

- ❑ создать UDP-сокеты (изготовить почтовый ящик)
- ❑ связать сокет с определенным номером порта и IP-адресом сетевого интерфейса (прикрепить почтовый ящик в определенном месте) – настроить адрес сокета.
- ❑ ОС принимает сообщения, пришедшие на этот адрес и складывает в сокет.
- ❑ дождаться поступления сообщения, прочитать, определить, от кого оно поступило и через какой сетевой интерфейс
- ❑ обработать полученную информацию и отправить результат по обратному адресу.



Связь между удаленными процессами с помощью датаграмм

Процесс-клиент:

- ❑ создать UDP-сокеты (изготовить почтовый ящик)
- ❑ связать сокет с определенным номером порта и IP-адресом сетевого интерфейса (прикрепить почтовый ящик в определенном месте) – настроить адрес сокета.
- ❑ положить сообщение в сокет, указав адрес назначения адрес сервера (IP и порт)
- ❑ прочитать из сокета ответ.



Создание сокета

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain,
           int type, int protocol);
```

Создание виртуального коммуникационного узла в ОС.

- ❑ **domain** – семейство протоколов для передачи:
 - PF_INET / AF_INET – семейство TCP/IP ;
 - PF_UNIX – семейство внутренних протоколов UNIX, т.н. UNIX domain.
- ❑ **type** – семантика обмена информацией
 - SOCK_STREAM – связь с помощью установления виртуального соединения;
 - SOCK_DGRAM – обмен через сообщения.
- ❑ **protocol** - конкретный протокол для выбранного семейства и способа обмена, если их несколько
 - = 0 для TCP/IP
- ❑ Возвращаемое значение
 - fd >= 0 – файловый дескриптор;
 - < 0 – ошибка, расшифровка в **errno**

Привязка сокета к адресу

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockd,
        struct sockaddr* my_addr,
        int addrlen);

struct sockaddr {
    short sa_family;
    char sa_data[14];
};
```

Привязка сокета к определенному полному адресу сети.

- ❑ **sockd** - дескриптор коммуникационного узла (сокета)
- ❑ **my_addr** – указатель на структуру с информацией о том, куда именно мы хотим привязать сокет – адрес сокета. Структура должна быть конкретизирована в зависимости от используемого семейства протоколов.
- ❑ **addrlen** - фактическая длина структуры с адресом. Может быть различной (например, для UNIX Domain).
- ❑ Возвращаемое значение
 - 0 – успех,
 - 1 – ошибка, расшифровка в **errno**

Адрес для семейства TCP/IP

```
#include <netinet/in.h>

struct sockaddr_in {
    short sin_family;
    unsigned short sin_port;
    struct in_addr sin_addr;
    char sin_zero[8];
};

struct in_addr {
    in_addr_t s_addr;
};
```

Адрес сокета для семейства TCP/IP

- ❑ **sin_family** – выбранное семейство, всегда **AF_INET**
- ❑ **sin_port** – 16-битовый номер порта в сетевом порядке байт; 0 – любой доступный
- ❑ **sin_addr** - адрес сетевого интерфейса.
- ❑ **sin_zero** – не используется, должно быть заполнено 0, может отсутствовать в разных реализациях
- ❑ **sin_addr.s_addr** – 32-битный IP-адрес интерфейса в сетевом порядке байт
 - INADDR_ANY** - все адреса локального хоста (0.0.0.0);
 - INADDR_LOOPBACK** - адрес loopback интерфейса (127.0.0.1);
 - INADDR_BROADCAST** - широковещательный адрес (255.255.255.255).

Преобразование порядка байт

```
#include <netinet/in.h>

unsigned long int htonl(
    unsigned long int host);
unsigned short int htons(
    unsigned short int host);

unsigned long int ntohl(
    unsigned long int net);
unsigned short int ntohs(
    unsigned short int net);
```

- ❑ **hton{l,s}** – перевод целого числа из порядка байт, принятого на компьютере, в сетевой порядок байт
- ❑ **ntoh{l,s}** – перевод целого числа из сетевого порядка байт в порядок байт, принятый на компьютере.

Преобразование IP адресов

```
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>

int inet_aton(const char *strptr,
             struct in_addr *addrptr);
```

Переводит символьный IP-адрес вида «192.168.0.1» в числовое представление в сетевом порядке байт

- ❑ **strptr** – строка символьного представления IP-адреса
- ❑ **addrptr** – указатель на структуру, куда помещается результат преобразования
- ❑ Возвращаемое значение:
1 – успех, 0 – ошибка (sic!)

Преобразование IP адресов

```
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>

char* inet_ntoa(struct in_addr
*addrptr);
```

Переводит символьный IP-адрес в числовом представлении в сетевом порядке байт в символьный вид «192.168.0.1»

❑ **addrptr** – указатель на структуру с адресом к преобразованию

❑ Возвращаемое значение:
указатель на строку с символьным представлением IP, при последующих вызовах заменяется

Отправка датаграмм

```
#include <sys/types.h>
#include <sys/socket.h>

int sendto(int sockd,
           char *buff, int nbytes,
           int flags,
           struct sockaddr* to,
           int addrlen);
```

Отправка *датаграмм* по протоколу UDP. Может блокироваться при отсутствии места в сетевом буфере.

- ❑ **sockd** — дескриптор сокета, через который отсылается информация.
- ❑ **buff** — адрес области памяти с информацией для передачи
- ❑ **nbytes** — количество байт, которое должно быть передано
- ❑ **to** — *адрес сокета* получателя
- ❑ **addrlen** — фактическая длина структуры с адресом
- ❑ **flags** — режим, 0 – по-умолчанию.
- ❑ Возвращаемое значение:
>= 0 количество переданных байт, < 0 - ошибка **21**

Приём датаграмм

```
#include <sys/types.h>
#include <sys/socket.h>

int recvfrom(int sockd,
             char *buff, int nbytes,
             int flags,
             struct sockaddr *from,
             int *addrlen);
```

Приём *датаграмм* по протоколу UDP. Блокируется при отсутствии пришедших датаграмм в сокете.

- ❑ **sockd** — дескриптор сокета, через который принимается информация.
- ❑ **buff** — адрес области памяти, куда помещается информация
- ❑ **nbytes** — макс. количество байт в буфере
- ❑ **from** — *адрес сокета* отправителя, может быть NULL
- ❑ **addrlen** — длина структуры с адресом. До вызова – максимальная длина, после вызова – фактическая длина
- ❑ **flags** — режим, 0 – по-умолчанию.
- ❑ Возвращаемое значение:
>= 0 количество принятых байт, < 0 - ошибка

Литература

1. В.Е. Карпов, К.А. Коньков - Основы операционных систем // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2192/31/info>
2. В.Е. Карпов, К.А. Коньков - Основы операционных систем. Практикум // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2249/52/info>
3. Google 😊

